Name: Aastha Srivastava
Class: B.Tech CSE (FSD)
Rollno: 2301350027

Date:
Page No.

SECTION - A

1) CPU registers are small, high-speed storage locations inside the CPU. They store data, address & instructions.

Importance:-
- store immediate data & instructions
- faster than cache & memory
- Improve execution speed of programs

eg: PC, IR, MAR, MDR, accumulator.

2) Program Counter (PC) holds the address of the next instruction to be executed. After fetching an instruction PC automatically increments. It ensures sequential execution of instructions. Updated during branch/jump ops.

3) Instruction format is the layout/structure of bits in an instruction. Defines how an instruction is represented in binary.

opcode	addressing mode bits	operand field

Main Field:-
- opcode: operation to perform
- operand fld/Register field: data location
- Addressing mode bits: how operand is accessed

4) Instruction encoding is defines how instruction are represented in binary form.
- Impact:
  - Instruction size
  - Decoding complexity
  - Processor performance.
- Efficient encoding: faster execution & less memory usage.

Date:
Page No.

5) Addressing modes defines how operands are accessed.

Purpose:
- Provide flexibility in accessing data.
- Reduce instruction count
- Improve program efficiency.

eg: Immediate A.M, Register A.M, Direct A.M etc.

6) Instruction Set Architecture (ISA): It is the set of instructions a CPU can understand & execute. It acts as an interface ~~bt~~ between hardware & software.

Include:-
- Instruction formats
- Data types
- Registers & addressing modes.

7) Types of instructions:-
1. Data Transfer: move data (LOAD, STORE)
2. Arithmetic: add subtract, multiply
3. Logical: AND, OR, NOT
4. Control flow: branch, jump, call
5. I/O instructions: input/output oper^n.

8) During Instruction decoding:-
- The control Unit (CU) reads the instruction from instruction Register (IR).
- It interprets the opcode (what oper^n to perform)
- Identifies:
  - Required oper^n
  - Operands / registers involved.
- Generates control signals to execute the inst^n.

Date :
Page No.

9) Basic Instruction Cycle consists of the following steps:-

1. Fetch: Instruction is fetched from memory using PC
2. Decode: Instruction is interpreted by CU.
3. Execute: Oper^n is performed (ALU or other unit)
4. Memory Access: Read/write from memory (if needed)
5. Write Back: Result stored in register/memory.

∴ This cycle repeats for every instruction.

10) Characteristics of RISC processors:-

- Simple & basic instructions.
- Fixed instruction length.
- Load/Store architecture (memory accessed only by load/store
- Large no. of registers.
- Efficient pipelining
- Faster execution due to simplicity.

11) Characteristics of CISC processors:-

- Complex instruction (can perform multiple operations)
- Variable instruction length.
- Uses microcode implementation
- Fewer instr^n needed per program
- More memory usage within instruction
- Slower pipelining

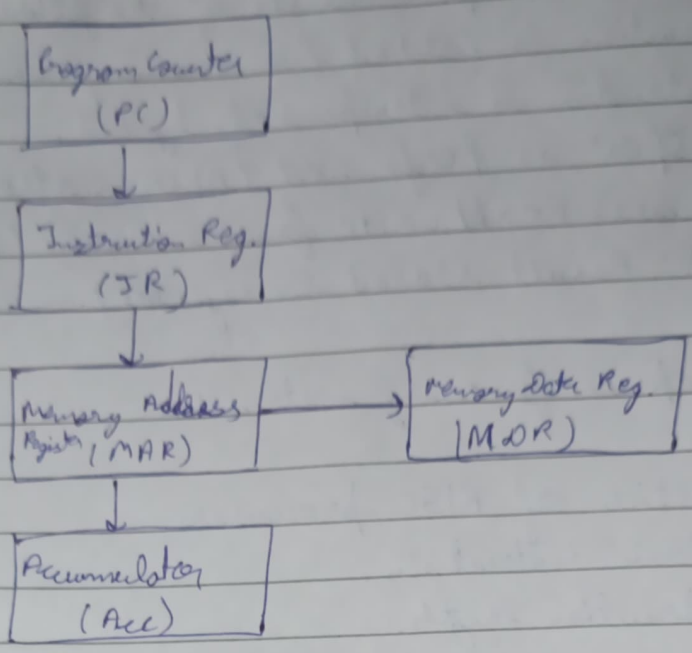
12)

Features	RISC-V	x86
Type	RISC	CISC
Instruction	Simple	Complex
Length	Fixed	Variable
Source	Open-source	Proprietary
Performance	Efficient pipelining	Complex decoding
Usage	Research, embedded systems	PCs, laptops.

Date :
Page No.

Section - B

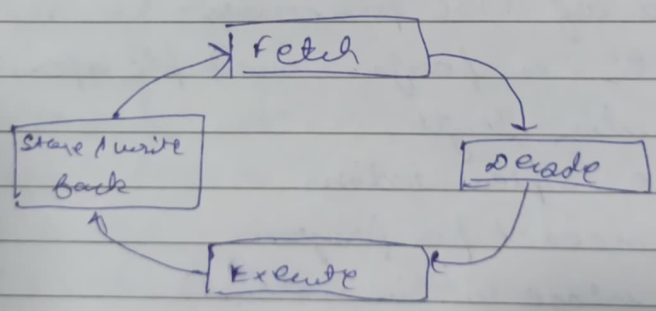
1)



Explanation:

- PC holds next instruction address
- IR stores current instruction
- MAR & MDR handle memory access
- Acc. stores intermediate results.

2)



- CPU fetches instruction, decode it, execute it & write back.

3) Instruction format

opcode	operand	Addressing mode

4)

Feature	RISC	CISC
Instruction	Simple	Complex
Instruction size	Fixed	Variable
Execution	Fast (1 cycle)	Multiple cycles
Memory Access	Load/store only	Direct memory access
Pipeline	Easy	Difficult.

Section-C

1) A program frequently uses memory addresses instead of registers
Impact on the performance:-
- Increased memory access time (memory slower than registers
- More CPU cycles required for each oper^n.
- Slower execution speed overall
- Reduced efficiency.

2) A processor designer wants simpler hardware & faster pipelining. They choose RISC architecture because:
- simple & fixed length inst^n.
- Easier to pipeline
- Requires simple hardware design.

3) A CPU inst^n contains opcode & 2 register operands
Execution (step-by-step):
1. Fetch: inst^n memory is fetched from memory.
2. Decode: opcode is interpreted.
3. Operand fetch: Data is read from registers
4. Execute: ALU performs the oper^n
5. Write Back: Result is stored in a register

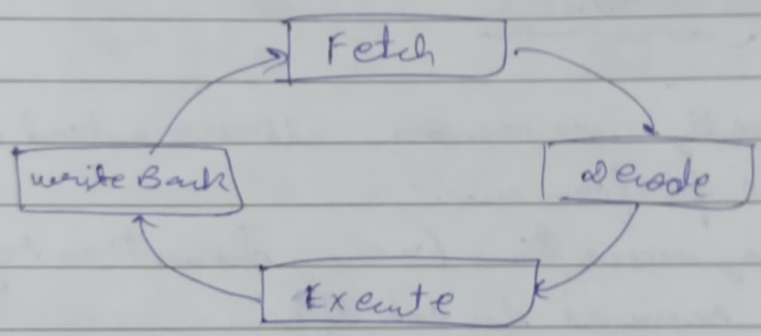
eg: ADD R1, R2 → R1 = R1 + R2

Date :
Page No.

4) X86 is widely used despite complexity because:
- Backward compatibility (supports old software)
- Large software ecosystem
- Powerful instr^n set
- Supported by major companies like intel & AMD.

SECTION - 2

1)



Explanation of fetch Stage:
- In the fetch stage, the CPU retrieves the instr^n from memory.
- The Program counter (PC) holds the address of the next instruction.
- This address is sent to memory via MAR.
- The inst^n is fetched into MDR, then moved to IR
- PC is incremented to point to next instruction

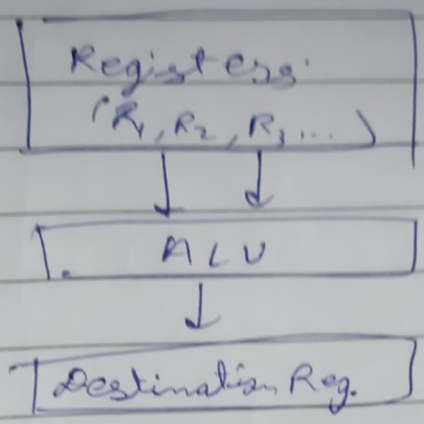
2)

funct 7	rs2	rs1	funct 3	rd	opcode

A typical RISC-V inst^n has:-
- opcode: specifies operation
- ~~rd~~ rd (destination register): where result is stored
- rs1, rs2 (source registers): Input operands
- funct3 & funct7: Define exact operation

Date :
Page No.

3)



Explanation:

- Data is read from source registers
- Sent to ALU for operation
- Result is written back to destination register
- Enables fast computation without memory access